Functions and scope

Module 2 - Lesson 1

Overview

Functions package logic under a name with parameters and return values. Scope rules decide which names are visible where, keeping your code organized and predictable.

Key Concepts

- def defines a function; parameters receive arguments; return sends a result back.
- Default arguments, keyword arguments, and *args/**kwargs give flexible call sites.
- Every function returns something; a missing return yields None.
- Local variables are created inside the function and vanish when it ends.
- Global scope is readable but mutable only via the global keyword; prefer passing data in.
- Type hints (def add(a: int, b: int) -> int) document intent and enable tooling.

Example

```
def area(width: float, height: float) -> float:
    return width * height

def describe(width, height, unit="m"):
    print(f"{width} x {height} {unit}")

describe(width=2, height=3)  # 2 x 3 m
```

Summary

Functions make code reusable and testable. Clear signatures, sensible defaults, and understanding of scope keep behavior predictable.

Practice Checklist

- Write a function with a default argument and call it with keyword arguments.
- Add type hints to a function and verify them with a type checker.
- Explain what a function returns when it has no return statement.